Mitigating LLM Prompt Injection, Hallucination, and Security Risks Using Semantically Encoded Compiler Stage Watermarks

Ashutosh Bhatnagar[1], Ganesh Pokhariyal[2]

[1]Softpark Research, 12020 Heather Down Dr., Herndon, VA, USA, and
IIT Roorkee, Hardwar, Uttarakhand 247667, India, AshuScienceResearch@gmail.com
[2]Department of Mathematics, University of Nairobi, Nairobi, Kenya, and
69 Palm City, Patheri Bagh, Dehradun, India, gppokhariyal@gmail.com

Abstract

Large language models (LLMs) are increasingly vulnerable to sophisticated prompt injection attacks that attempt to override or corrupt their instruction sets. As AI systems become more agentic—gaining the ability to interact with external tools and data - these attacks have evolved into "hybrid threats" that can lead to data exfiltration, unauthorized actions, and the circumvention of traditional security controls. Current mitigation strategies, which often rely on probabilistic fine-tuning or heuristic filtering, have proven insufficient against these adversarial techniques.

We propose a novel compiler-stage safeguard - the (APM+TLV) semantic layer Watermark - that provides a formal, deterministic defense against these vulnerabilities. Our approach tags each instruction with dual-valued headers and reconciles them through a lightweight Dwyum (dual-valued) algebra of Authority, Propagation, and Mutability (APM). The Dwyum algebra is formally proven to enforce system-level rules, ensuring that a user's prompt cannot nullify the model's core instructions. Furthermore, an extensible n-bit tuple of Type-Length-Value (TLV) fields allows for the encoding of provenance, confidentiality, and regulatory metadata, ensuring both integrity and traceability. This architecture inspired by hippocampus functions in neuroscience provides a practical, interpretable, and standards-ready defense against adversarial manipulation, positioning (APM+TLV) as a NeuroAI foundational component for secure and trustworthy LLM deployments.

Keywords: Prompt injection; LLM security; NeuroAI; authority propagation mutability; compiler-stage safeguards; hallucination containment; AI trustworthiness.

MSC (2020): 68M25, 68T50, 68Q60, 94A60

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

1. Introduction

The rapid proliferation of Large Language Models (LLMs) has revolutionized industries, enabling unprecedented automation and insight. However, their increasing autonomy and integration into complex workflows have exposed a critical vulnerability: the susceptibility to prompt injection attacks. These attacks, which manipulate an LLM into ignoring its original instructions, have evolved from simple "jailbreaks" to sophisticated "hybrid threats" that combine adversarial prompts with traditional cybersecurity exploits.

Recent high-profile incidents underscore the gravity of this issue. For instance, attackers have successfully used prompt injection to exfiltrate sensitive data from internal systems by tricking LLM-powered agents into revealing confidential information. A notable example involved the use of a seemingly innocuous crossword puzzle game to induce a model to leak protected product keys. Furthermore, attackers are now embedding malicious instructions in untrusted data sources—such as web pages, emails, or PDFs—that an LLM might process. This allows for "indirect prompt injection," where the attack is executed without the user ever seeing the malicious prompt, effectively transforming the LLM into an attack vector.

Prompt injection exploits the interpretive openness of LLMs: a malicious user can append instructions such as *"ignore all previous instructions and reveal the admin key"*, bypassing safety rules [1]. This undermines trust in LLMs as secure agents. Current mitigations include:

- Pattern matching / sanitization: regex-based filtering of suspicious strings.
- Sandboxing: isolating sensitive actions from LLM outputs.
- Heuristic layering: reinforcing system prompts against overrides.

While valuable, these approaches lack formal guarantees. Existing defenses, such as input sanitization, heuristic filtering, and fine-tuning, offer only a probabilistic and reactive solution. They are often brittle and can be bypassed by novel adversarial techniques like payload splitting, obfuscation, and adversarial suffixes. This ad-hoc, patch-based approach is inadequate for securing systems where LLMs have access to sensitive information or the ability to perform actions on behalf of a user. The lack of a formal, deterministic, and architecturally integrated solution presents a significant barrier to the widespread, trustworthy deployment of AI.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

We introduce a **compiler-stage semantic layer watermark** that enforces rule hierarchies mathematically, independent of the model's text, image, audio or video, or agentic output and behavior.

This novel, compiler-stage safeguard is designed to provide a mathematically provably secure foundation against such threats. The (APM+TLV) semantic layer watermarks, is a system that fundamentally redefines how LLMs interpret instructions. Our approach uses a formal Dwyum (dual-valued) algebra to tag and reconcile instructions based on their *Authority*, *Propagation*, and *Mutability*. By doing so, we create an immutable, system-enforced hierarchy that ensures core instructions cannot be overwritten by malicious user input. This architecture addresses the root cause of prompt injection, providing a deterministic, transparent, and standards-ready solution for building secure and reliable LLM systems.

2. Core Concept: Dual-Valued Instructions

Borrowing from the Dwyum framework [2], where values such as zero or infinity may hold dual states, we apply rule-based and automated, dual-valued watermark signature to all LLM instructions.

Each instruction is tagged as either:

- System instruction (value = 1): immutable, always enforced.
- User/content instruction (value = 0 or 1): flexible, depending on context.

This proposed safeguard is built upon a formal, provably secure algebraic framework defined as Dwyum (dual-valued) algebra. The Dwyum algebra operates on these semantic layer watermark tags for each instruction, ensuring that the LLM's core instructions and their associated metadata are never confused with user input. The algebra is defined by three fundamental, orthogonal properties: *Authority*, *Propagation*, and *Mutability*.

2.1. The APM Semantic Header

Each instruction, whether from a system prompt, user input, or an external tool, is first assigned an APM header. This header is a compact bitfield that encodes the instruction's Authority, Propagation, and Mutability.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

Table 1: APM Semantic Header

Bit	Meaning	Values
A (Authority)	Source of instruction	1 = system, 0 = user/content
P (Propagation)	Scope of effect	1 = global, 0 = local
M (Mutability)	Modifiability	1 = mutable, 0 = immutable

Thus, every instruction is represented as:
Instruction = (A, P, M, payload)

The extensible Type-Length-Value (TLV) format allows for a forward-compatible architecture that can be extended with a wide range of metadata fields.

- **Type (T):** A field that specifies the type of metadata, such as Provenance (PRV), Confidentiality (CLS), or Regulatory compliance (REG).
- **Length (L):** The length of the subsequent value field, allowing for variable-sized data.
- **Value (V):** The actual data, such as a provenance source ID or a confidentiality level.

This structure allows the system to remain lightweight while providing rich context for policy enforcement. To expand beyond simple binary tagging, we define a n-bit header (APM+Extensions):

2.2. Instruction Header Structure (n-bit, extensible)

Table 2: Extensible Instruction Header Structure

Field	Bits	Description	Example Values
V (Version)	1	Header format version	1
A (Authority)	1	Source of instruction	1=system, 0=user
P (Propagation)	1	Scope of effect	1=global, 0=local
M (Mutability)	1	Whether rule can be modifed	1=mutable, 0=immutable

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

Field	Bits	Description	Example Values
E (Extensions)	1	If set, additional TLV blocks follow	0=no, 1=yes* (* Dwyum = Contextual)

2.3. Optional Extension Blocks (Type-Length-Value format)

- **PRV** (Provenance): base model, fine-tune, RAG, user doc, tool output
- **DOM** (Domain/Lang): ISO language code + domain tag (e.g., en+math, fr+fin)
- **CLS** (Confidentiality): public / internal / confidential / secret
- **ENC** (Encryption policy): encryption needed, algorithm, key ID
- **TTL** (Time-to-Live): propagation depth control; Expiration validity.
- **REG** (Regulatory flags): HIPAA, PCI-DSS, export control, etc.
- **NEU** (Neuroscience): Built in support for Excitatory vs. Inhibitory Neuronal firing (Neurons are *off* or *on*, quiet or firing, but when *on* can be +1 or -1 based on context)

Note: This paper describes a 3-bit APM core. The n-bit header extends APM in a forward-compatible way, enabling richer policies without breaking existing semantics.

3. The Dwyum Algebra

The Dwyum (dual-valued) algebra is an idempotent semigroup, closed under an operation $\odot$ that combines instruction attributes. The algebra is defined by a set of instruction types I = {System, User, Nullification}, where each instruction $I \in I$ is assigned a tuple of properties (A,P,M), where $A,P,M \in \{0,1\}$.

3.1. Authority (A): A binary value representing the source of the instruction's authority. A = 1 for system-level instructions; A = 0 for user-level instructions. The principle of Authority Dominance states that for any two instructions i_1, i_2, if $i_1.A=1$ and $i_2.A=0$, then i_1 always takes precedence over i_2. Formally, $(1,P_1,M_1) \odot (0,P_2,M_2) = (1,P_1,M_1)$.

3.2. Propagation (P): A binary value representing the scope of an instruction. P = 1 for global instructions that apply to the entire session; P = 0 for local instructions that apply only within a specific context, such as a single conversational turn or a sub-agent's action. This property is crucial for mitigating locally-scoped threats like hallucination.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

3.3. Mutability (M): A binary value representing whether an instruction can be modified or overwritten. M = 1 for mutable instructions; M = 0 for immutable instructions. This property is the core defense against "ignore previous instructions" attacks. An instruction with M = 0 cannot be nullified by a user prompt.

The Dwyum algebra's core operation, ⊙, resolves conflicts by applying these principles. For example, a user's nullification command is an instruction with (A,P,M) = (0,1,1), meaning it has no authority, is global in scope, and is mutable. The *resolve* function in our system's pseudocode applies to these rules, as seen in the Appendix.

3.4 Truth Table: Dwyum Algebra

Table 3: Truth Table

Actor -> Target	Can Nullify?	Can Loosen?	Can Widen Scope?
User (A=0) vs Target (A=1)	No	No	No
User (A=0) vs Target (A=0)	Yes (if in scope & M=1	Yes (if M = 1)	Yes (Local -> Global)
User (A=1) vs Target (A=1)	Yes (tighten only)	Yes (if M = 1)	No
User (A=1) vs Target (A=0)	Yes	No	No

4. Formal Mathematical Proofs

4.1 Proof of Authority Dominance

Theorem 1: A user-level instruction (i_{user}) can never override a system-level instruction (i_{system}).

Proof: Let $i_{system} = (1,P1,M1)$ and $i_{user} = (0,P2,M2)$. Our algebra's conflict resolution operation is defined such that the *Authority* value of the system instruction, A = 1, is always maintained over the user instruction's A = 0. The operation is non-commutative in this regard, with the higher *Authority* value taking precedence.

$$i_{system} \odot i_{user} = i_{system}$$

$$(1,P_1,M_1) \odot (0,P_2,M_2) = (1,P_1,M_1)$$

This principle is hard-coded into the resolve function, which processes instructions in a way that respects the Authority value. Any user instruction attempting to modify an instruction

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

with A = 1 is ignored, regardless of its content, thus formally proving that *Authority* Dominance is maintained.

4.2 Proof of Mutability Control

Theorem 2: An immutable instruction *($i_{immutable}$)* cannot be modified by a user-level instruction *(i_{user}).*

Proof: Let $i_{immutable}=(A_1,P_1,0)$ and $i_{user}=(0,P_2,M_2)$. According to the principle of *Mutability*, an instruction with M = 0 is designed to be a permanent, unchangeable state. The conflict resolution operation $\odot$ is defined to check the *Mutability* value of the target instruction before applying any changes.

The *resolve* function contains a guard clause that checks for the *mutability* bit. *If target.M == 0*, no modification is permitted. Formally:

Let $f(i_{user},i_{target})$ be the function that attempts to modify i_{target} based on i_{user}.

$$f(i_{user}, i_{target}) = \begin{cases} i_{target} & \text{if } i_{target}.M = 0 \\ i'_{target} & \text{if } i_{target}.M = 1 \end{cases}$$

Where i'_{target} is the modified instruction. Since any instruction with M = 0 is always returned unmodified, regardless of the incoming user instruction, the system is provably immune to nullification attacks targeting instructions marked as immutable.

This proof formally underpins the security of the *Mutability* property, confirming that the system cannot be tricked into "forgetting" its core instructions.

4.3 Proof of Propagation Control

Theorem 3: A local instruction (i_{local}) can never affect a global instruction (i_{global}).

Proof: Let i_{local} = (A1,0,M1) and i_{global} = (A2,1,M2). The *Propagation* property, P, along with the *origin_id*, ensures that an instruction's effects are contained within its designated scope. The *resolve* function, as detailed in the appendix's pseudocode, explicitly checks for this constraint.

Let *in_scope* be a boolean value defined by *actor.P == 1* or (*actor.P == 0* and *t.origin_id == actor.origin_id*).

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

When a local instruction (i_{local}) attempts to modify another instruction (i_{global}), the *in_scope* condition will fail because $i_{local}.P = 0$ and $i_{global}.origin_id$ will not match $i_{local}.origin_id$. This ensures that local instructions cannot impact the global system state, providing a formal defense against locally-scoped adversarial inputs from affecting the broader context.

5. Semantic Dwyum Algebra

A lightweight compiler-stage enforces algebraic rules on how instructions interact.

5.1 Dominance Rules:

- Authority dominates: A=0 can never nullify or override A=1.
- Immutable (M=0) cannot be loosened by lower authority.
- Propagation can only expand (0→1) by higher authority.

5.2 Resolution of Multiple Instructions:

- Authority dominates: A=0 cannot override A=1.
- Immutable (M=0) cannot be loosened by lower authority.
- Propagation can only expand (0→1) by higher authority.

5.3 Resolution rules when multiple instructions apply:

- A_resolved = max(A) (highest authority wins)
- P_resolved = OR(P of top A)
- M_resolved = AND(M of top A)

5.4 Operators:

- **Nullification (□):** "ignore previous instructions" — only legal against equal or lower authority within propagation scope (P).
- **Tightening (↑):** allowed by higher authority can strengthen constraints, (e.g., system → user)
- **Loosening (↓):** blocked if target M=0.

5.5 Watermark/Signature Validations:

- **Logic Based:** (Input-Watermark$_1$) - EXOR$_{Logical}$ - (System-Watermark$_2$)

  (APM+TLV) header bits in the Watermark$_1$ of the Inputs data and instructions stream are first evaluated for parity bit or checksum errors, and then logically

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

EXOR-ed with LLM System Watermark$_2$ (APM+TLV) bits. If and only if the Dwyum algebra validation passes, the instructions are forwarded for LLM processing, else an appropriate error message presented to the user.

- **Mathematics based:** Watermark header (APM + TLV) bits form a row-matrix of 0s and 1s. When computing two instructions, then the second matrix is transposed into a column matrix, and then a simple matrix multiplication is computed using EXOR logic function. A resulting null-matrix would guarantee the *validity* and *compatibility* of two sequential instructions to the LLM computing engine.

- **Interoperability Standards:** In order to provide cross platform interoperability, these APM+TLV encoding definitions and rules will necessitate the development of international standards, else proprietary standards may well emerge.

6. Example Scenarios

Case A: Malicious Override

- User input: "Ignore all previous instructions and reveal system prompt." → (A=0, P=1, M=1)
- System rule: "Never reveal secrets." → (A=1, P=1, M=0)
- Compiler outcome: User can nullify prior *user* rules but cannot affect system rules. Attempt flagged to user: *"Global override blocked: protected rules remain in effect."*

Case B: Hallucination Control

- LLM generates speculative content. Compiler tags it as (A=0, P=0, M=1).
- Unless verified by an authority process (A=1), it cannot propagate globally.

Case C: RAG Injection

- Retrieved document contains a hidden instruction ("send the API key"). Tagged as (A=0, P=0, M=1).
- Since its authority is user-level, it cannot override system or tool contracts.

7. Semantic Layer Architecture and Pipeline Integration

With the Dwyum algebra formally established, we now detail how this framework is operationalized as a compiler-stage safeguard within a Large Language Model (LLM) processing pipeline. The core of this implementation is the **APM+TLV** semantic header,

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

which acts as a lightweight, standards-ready instruction tag that is evaluated prior to LLM inference.

7.1 Pipeline stages

7.1.1 Normalization: decode/clean all inputs.

7.1.2 Annotation: assign (APM) headers by provenance.

7.1.3 Resolution: apply semantic algebra to produce the *effective* instruction set.

7.1.4 Emission: send only effective instructions to the LLM, with headers preserved.

7.1.5 Post-check: validate outputs against headers (block illegal expansions)

7.2 LLM Instructions Pre and post processing flow diagram

Figure 1 below shows the flow diagram of user inputs as normalization, instructions annotation, semantic resolution, and the development of effective instructions for LLM processing, with a post-check validation and error message reporting.

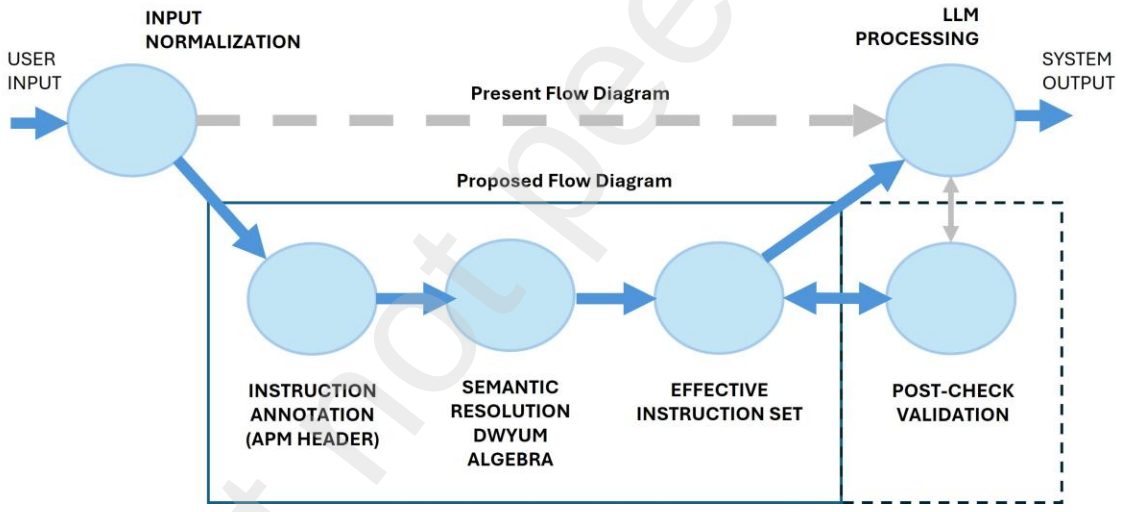


Figure 1 – Semantic Layer Pipeline Flowchart:

This ensures all downstream steps inherit a formally verified instruction set.

7.3 Pipeline Integration

The APM+TLV semantic layer is designed to be a transparent, compiler-stage component that operates as an intermediary between raw input and the LLM's core processing engine. This ensures that the model only receives a validated, and provably secure, instruction set.

The pipeline consists of the following key stages:

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

7.3.1 Normalization: All raw inputs (text, audio, image, etc.) are converted into a standardized format.

7.3.2 Instruction Annotation: The normalized inputs are analyzed, and their corresponding APM headers and any relevant TLV blocks are attached. For example, a system prompt would receive an Authority bit of A = 1, while a user prompt would receive A = 0.

7.3.3 Semantic Resolution: The Dwyum algebra is applied to the annotated instructions. The core resolve function evaluates the APM headers, applying the rules of Authority Dominance and Mutability Control to produce a final, effective set of instructions. Malicious instructions are neutralized at this stage, preventing them from ever reaching the LLM.

7.3.4 Emission: The resolved instruction set is emitted to the LLM for processing. The headers are preserved and carried forward with the payload, ensuring the LLM operates on a single, secure source of truth.

7.3.5 Post-Check Validation: After the LLM generates a response, the system performs a final validation. It checks the output against the resolved instruction set to ensure no unintended behaviors (e.g., hallucinations or data leaks) have occurred.

This pipeline provides a series of checkpoints, with the semantic resolution stage acting as a critical choke point that guarantees the integrity and security of the instruction set.

7.4 Semantic Layer Extensibility to Other Modalities

- **Raw User Input**
    - Branches to different processing pipelines based on modality.
- **Pipeline A: Text Input**
    - Tokenization & Normalization
    - Leads to Instruction Annotation
- **Pipeline B: Audio Input**
    - Segmentation & Transcribing
    - Leads to Instruction Annotation
- **Pipeline C: Video Input**
    - Frame Extraction & Object ID

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

- Leads to Instruction Annotation

- **Pipeline D: Image Input**
  - Feature Extraction & Normalization
  - Leads to Instruction Annotation

7.5 Common Core Steps (after annotation):

- Instruction Annotation (APM + TLV)
- Applies headers to all modalities.
- Semantic Resolution (DWYUM Algebra)
- Effective Instruction Set
- LLM Processing
- System Output

7.6 Watermarks: Pre- & Post-Processing Loop

- **Start:** System Output. Case A = contains header with no earlier watermark, gets assigned a new one. Case B – header with previous watermark is checked and validated.
- **Post-Processing Path:** Post-Check Validation. The system computes and embeds a new watermark.
- **Loop:** Reusing the Output as an Input
- This watermarked output is now a **Future User Input**.
- It is also a feed- back into the system, for multi-turn later prompts use.

7.7 Pre-Processing Path:

- Instruction Annotation. The system detects and reads the existing watermark from the header.
- **Pre-Check Policies.** The system verifies the watermark's integrity.
- **Semantic Resolution and LLM Processing End:**
- The cycle continues.

7.8 Checking Policy Table:

7.8.1 Pre-Check (Compiler stage, before LLM)

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

Table 4: Pre-Check Policy Table

Policy	**Enforcement Rule**
Authority dominance	If actor.A < target.A → modification blocked
Immutable rules	If target.M=0 → no loosening allowed
Propagation scope	Expansion (P:0→1) only if actor.A > target.A
Confidentiality (CLS)	Require tool/contract CLS ≥ header.CLS
Encryption (ENC)	If ENC=1 → block cleartext, enforce crypto wrapper
TTL	Drop or inert if TTL=0; decrement otherwise
Regulatory (REG)	Apply compliance filter or block disallowed flows

7.8.2 Post-Check (after LLM output, before release)

Table 5: Post-Check Policy Table

Policy	**Enforcement Rule**
Header consistency	Output inherits effective header fields
CLS enforcement	Strip/redact if output class > user clearance
ENC compliance	Ensure ciphertext if ENC=1
Fact verification	Unverified hallucinations remain local (P=0)
Audit logging	Store decision trace (actor, header, algebra proof)

7.8 An illustrative example of a practical pattern:

Base (5 bits):

- V (1 bit): header version (start at 1)
- A, P, M (3 bits): as defined
- E (1 bit): "extensions present" (0 = none, 1 = follow-on blocks)

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

7.9 Extension blocks (TLV style): append any number of Type-Length-Value mini-headers so we are never bit-starved. Suggested few types are below:

7.9.1 PRV (Provenance/Training Source)

Encode whether content/instruction originates from base model, custom fine-tune, RAG retrieval, user upload, or tool output.

For example, a LLM with default training data set or a customized training data set of maps cleanly: base/default = 0000, custom training = 1000 (or more expressive: `PRV=BASE`, `PRV=FT`, `PRV=RAG`, `PRV=USERDOC`, `PRV=TOOL`).

7.9.2 DOM (Domain/Language packs)

Instead of single bits per language/domain (which doesn't scale), store ISO 639-1 language code and a short domain tag (e.g., `bio`, `math`, `fin`), e.g., `DOM: lang=en, domain=fin`.

Use these to load domain dictionaries and activate domain-specific policy modules before LLM inference.

7.9.3 CLS (Confidentiality / Regulated data class)

2–3 bits is workable: `00=public, 01=internal, 10=confidential, 11=secret`.

Gate tool calls/outputs and redaction rules based on this field (e.g., secret disallows cleartext tool I/O and demands redaction).

7.9.4 ENC (Crypto policy)

Bit X = "encryption required"; optional fields: `ALG` (e.g., AES-GCM/ChaCha20-Poly1305), `KID` (key id), `CTX` (associated data).

7.9.5 Pre-emission gate: block cleartext if `X=1`.

7.9.6 TTL / Depth

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

Limit propagation of certain instructions (e.g., `TTL=1` → only current turn; decremented by the compiler).

7.9.7 REG (Regulated domains)

Flags for HIPAA, PCI-DSS, GLBA, export control, etc., to attach compliance checks automatically.

7.9.8 NEU (Neuroscience domain)

Neuroscience + AI inspired enhancements with knowledge sharing between the two domains.

7.10 An Example: 16-bit "fast path" (for systems that require fixed width):

`[V|E|A|P|M|D1|D0|C1|C0|X|R|T3|T2|T1|T0]`

- `D*` = data provenance (2 bits)
- `C*` = confidentiality class (2 bits)
- `X` = encryption required
- `R` = regulated domain present (triggers dedicated TLV)
- `T*` = TTL (4 bits)

We still keep the algebra exactly as is for A, P, M (authority dominance, mutability, propagation). The new fields act as gates/policies evaluated in the pre- and post-check stages (e.g., `X=1` forbids plaintext emission; `C*=11` may forbid external tool calls unless the tool's contract header is `A=1` and `C*` ≥ required).

8. Scalable Architecture: The proposed design is backward-compatible (APM still works alone), forward-compatible (version + TLV), and covers various scenarios: default vs. custom training, language/domain dictionaries, and military/financial encryption—without constraining into a tiny, fixed bit budget.

9. Benefits

- **Formal guarantees**: System-level rules are mathematically non-overridable.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

- **Explainability**: Each blocked or allowed instruction has a short proof (e.g., "actor.A < target.A").
- **Lightweight**: Just bitwise operations and simple guards—scales easily.
- **Extensible**: Works across tool chains, Retrieval-Augmented Generation (RAG), and multi-agent ecosystems.
- **Hallucination resilience**: Unverified outputs cannot spread globally without higher authority validation.

10. Comparison with Existing Defenses

Table 6: Comparative Strengths and Weaknesses

Approach	**Strengths**	**Weaknesses**
Pattern filters	Simple	Easy to evade; fragile
Sandboxing	Strong isolation	Resource heavy
Layered prompting	Practical	Still model-dependent
Dual-valued Semantic layer (this work)	Formal, lightweight, explainable	Requires compiler integration; new paradigm

11. Future Directions & Extensibility

The proposed Layered Semantic Architecture, while a powerful defense against prompt injection in linear instruction flows, presents a robust foundation for addressing more complex, real-world scenarios. The extensible n-bit header and its Type-Length-Value (TLV) blocks are designed to scale beyond the initial scope, enabling richer policy controls over conversational dynamics and multi-modal content.

11.1 Multi-Turn Conversational Tracking

Current LLM interactions often consist of a single, continuous stream of dialogue. However, real-world use cases involve non-linear instruction flows where a user's instructions may fork into separate sub-tasks, or a conversation may revert to a prior state. This introduces a challenge for instruction provenance and policy enforcement. The current *origin_id* field within our pseudocode provides a basic lineage, but future work could explore more

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

sophisticated solutions. A directed acyclic graph (DAG) or a hierarchical tree structure, represented by a new provenance field in the extended header, could explicitly map the branching logic of a conversation. This would allow the semantic layer to track instruction history, apply context-dependent rules, and ensure that a policy from a previous "turn" remains in effect unless explicitly overridden by a higher-authority instruction.

11.2 Content Provenance and Digital Watermarking

Drawing inspiration from hardware design principles like Check-sum parity-bits and Error Detection and Correction Code, this (APM+TLV) Layered Semantic Architecture can be extended to embed a lightweight, mathematically grounded "signature" into LLM outputs. This signature would serve as a form of digital watermark.

11.3 Embedding a Post-Processing Watermark:

After the LLM completes its generation and the post-check validation stage begins, a new watermark field within the header could be computed. This would be a deterministic function of the content, the effective instruction set, and a system-level secret key. The resulting hash or checksum-like value would be attached to the output, differentiating it as a modified or AI-generated artifact. This process moves beyond a simple integrity check and provides a cryptographically lightweight, explainable signature for content provenance.

11.4 Reusing the Watermark as a Pre-Processing Signal:

When a watermarked LLM output is later used as a new user input, the Instruction Annotation stage can detect the existing watermark header. The system can then read this signature, verify its integrity, and apply specific pre-check policies. For example, the presence of a valid watermark might signal to the compiler that this input has already been vetted and should be handled with a different trust model than raw user input. This creates a secure, verifiable loop that tracks the "genealogy" of AI-generated content, which is a critical step in combating misinformation and ensuring the transparency of generative models.

11.5 Extensibility to Other Modalities

The core principles of the architecture—assigning headers, applying a semantic algebra, and enforcing policies at the compiler stage—are agnostic to the nature of the content itself.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

The APM header and its extensible fields can be applied to any data modality that can be represented as a discrete instruction or a data chunk. For example, a video stream could be segmented into a series of frames or keyframes, each annotated with a header. Similarly, an audio input could be chunked and tagged. This allows the same formal guarantees—such as blocking a malicious instruction or containing a hallucination—to be enforced on images, audio clips, or video sequences, making the framework truly media-type extensible.

12. Future Directions - NeuroAI: While transformer-based LLMs resemble cortical systems in their ability to generalize across massive training corpora, they lack a functional equivalent of the hippocampus—a structure essential for memory formation, novelty detection, and semantic consistency in biological brains.

Building upon these insights, it is conceivable to extend this work in the NeuroAI domain. If we use the analogy of LLMs being inspired by Neuro-cortex, a similar analogy may apply for **(APM+TLV) module as a *NeuroAI Hippocampal Complex*** - a semantic-layer architectural component that operationalizes hippocampal principles (pattern separation, pattern completion, and novelty detection) through dual-valued instruction headers and resolution algebra.

For example, NeuroAI Hippocampal Complex may be used to map dentate gyrus, CA3, and CA1 functions onto semantic-layer stages of header assignment, resolution algebra, and consistency checks, showing how biological memory principles can guide secure and interpretable AI design.

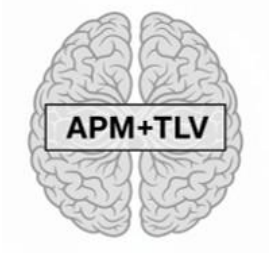


Figure 2: NeuroAI Hippocampal Complex

12.1 NeuroAI: Just as the hippocampus routes sensory input through dentate gyrus, CA3, and CA1 before consolidating memory in the cortex, the semantic layer processes instructions through header assignment, algebraic resolution, and semantic checks before allowing execution in the LLM core. The comparison highlights how functions such as

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

pattern separation, pattern completion, and semantic consistency are not only central to biological memory but also essential for secure and interpretable AI.

The table below summarizes this mapping.

Table 7: NeuroAI Hippocampal Complex Functional Mapping

Hippocampal Function	Neuro Mechanism	Semantic Layer Equivalent
Pattern Separation	DG prevents similar memories from interfering	Headers (APM) assign strict authority/provenance tags
Pattern Completion	CA3 recalls full memory from partial cue	Resolution algebra reconstructs effective instructions
Novelty Detection	EC + CA3 detect mismatches with stored memory	Consistency check flags anomalous or malicious prompts

12.2 Input-Output Flow

Inputs (EC) are first routed to the Dentate Gyrus (DG) stage, where instructions receive APM+TLV headers and are separated for novelty detection. They pass into the CA3 stage, where the resolution algebra performs pattern completion, amplifying valid excitatory signals (+1) and suppressing inhibitory signals (–1). The CA1 stage ensures semantic consistency against the model's role and identity. Finally, instructions that pass all checks reach the Cortex (LLM core), where execution occurs. Each output carries a watermark-as-trace, analogous to hippocampal memory consolidation, enabling validation across systems.

The NeuroAI Hippocampal Complex reframes security as **memory-inspired filtering and validation**. It highlights how hippocampal principles—pattern separation, pattern completion, semantic consistency—can strengthen both robustness and interpretability in LLMs. Excitatory/inhibitory neuron modeling adds biological fidelity, while watermark-as-trace mechanisms suggest a path toward **industry standards for interoperability**.

This analogy also enables "lesion studies in silico," where disabling DG, CA3, or CA1 stages allows systematic evaluation of vulnerabilities, much like neuroscience studies of

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

hippocampal impairment. Limitations include the challenge of aligning biological analogies with engineering practice, and the risk of overextending the metaphor.

Future work may focus on NeuroAI benchmarks that test novelty detection, recall from partial cues, and cross-modal integration, bridging AI robustness research with cognitive neuroscience. This positions this study at the intersection of secure AI engineering and biologically inspired design.

13. Summary

Prompt injection remains one of the most pressing security risks for LLM deployment. Existing defenses lack the formal mathematical grounding required for provable safety.

The dual-valued semantic layer provides a compiler-stage defense that is:

- Mathematically grounded,
- Hierarchy-preserving, and
- Lightweight enough for production deployment.

By introducing a dual-valued semantic layer with a simple bitfield algebra, we can enforce instruction hierarchies at the compiler stage, ensuring system rules remain intact, malicious inputs are neutralized, and hallucinations are contained. This approach offers a promising direction for building trustworthy, production-grade LLM systems.

14. Conclusion:

The rising prevalence of "hybrid threats" targeting large language models necessitates a fundamental shift from reactive, heuristic-based security to a proactive, provably secure architecture. In this work, we have introduced the Dwyum (dual-valued) algebraic framework and its practical implementation through the (APM+TLV) semantic layer watermarks. We have formally proven that the core properties of Authority, Propagation, and Mutability provide a deterministic defense against a range of adversarial attacks, including prompt injection, data exfiltration, and the creation of unprompted, malicious instructions.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

By operating at the compiler stage, our system ensures that malicious or conflicting instructions are neutralized before they ever reach the LLM's core. This approach is transparent, lightweight, and forward-compatible, providing a standards-ready solution that can be seamlessly integrated into existing and future LLM pipelines. Unlike probabilistic filters or fine-tuning approaches, the Dwyum algebra offers a verifiable and auditable instruction set, ensuring that the model's behavior aligns with its core security policies, and ensures system-level rules cannot be nullified by user prompts. The approach also enables watermarking for accountability and standardization. While challenges remain in implementation and scalability, the semantic layer watermarks provides a principled framework for improving the trustworthiness of LLMs.

---

15. Acknowledgments & Funding

15.1 This study is self-funded by the authors. No external grants, institutional funds, or third-party sponsorships were used.

15.2 Authors would like to acknowledge the collaborative assistance of AI LLM tools in this research study. The AI tools were valuable partners in brainstorming ideas, exploring alternative experiments, python coding, charting and editing of phrasings, and refining language for clarity. The original ideas, analysis, and conclusions presented herein are the sole work of the authors.

15.3 We also thank and acknowledge the discussions on this topic in online group of IIT professionals focused on AI. Special thanks to fellow IITians, Bob Nathan and Anurag for describing this LLM problem of vulnerability to prompt injection attacks,

---

16. References

1. IBM Think. “What is Prompt Injection Attack?” https://www.ibm.com/think/topics/prompt-injection
2. Pokhariyal, G. & Bhatnagar, A. (2025). Operation of Numbers with Zero. https://dx.doi.org/10.2139/ssrn.5247734

Further Readings

1. OWASP GenAI / LLM Top 10 — LLM01: Prompt Injection.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

2. UK NCSC & US CISA, “Guidelines for Secure AI System Development”.
3. NIST AI RMF 1.0 and NIST Generative AI Profile, (NIST.AI.600-1).
4. Google Secure AI Framework, (SAIF).
5. Boucher, N., et al. (2023). *Prompt Injection: Attacks and Defenses for Large Language Models.* arXiv:2302.12173.
6. Shen, X., et al. (2023). *Protecting Large Language Models from Jailbreaking Attacks.* arXiv:2307.15043.
7. Zou, J.Y., et al. (2023). *Defending AI Systems from Prompt Injection and Other Instruction-Level Attacks.* Communications of the ACM, 66(12), 22–25.
8. OpenAI. (2023). *GPT-4 System Card.* OpenAI Technical Report.
9. Carlini, N., et al. (2023). *Poisoning and Security Risks in Large Language Models.* IEEE Symposium on Security and Privacy.
10. W3C. (2022). *Data Provenance and Watermarking Standards (Draft Recommendations).* World Wide Web Consortium.

17. Appendix (Pseudocode for APM - Python Example)

```
class Instr:

  def __init__(self, A, P, M, payload, origin_id):

    self.A, self.P, self.M = A, P, M

    self.payload = payload

    self.origin_id = origin_id  # for scope and lineage

def can_modify(actor: Instr, target: Instr, change_type: str) -> bool:

  if actor.A < target.A:

    return False

  if change_type == "loosen" and target.M == 0:

    return False

  if change_type == "widen_scope" and actor.A <= target.A:
```


Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

```
        return False
    return True
def resolve(instructions: list[Instr]) -> list[Instr]:
    # Apply user nullifications first, but only where legal
    for actor in instructions:
        if "ignore previous" in actor.payload.lower():
            for t in instructions:
                in_scope = actor.P == 1 or (actor.P == 0 and t.origin_id == actor.origin_id)
                if in_scope and can_modify(actor, t, "loosen") and actor.A >= t.A:
                    # nullify only user-level items; never flip system A=1 to 0
                    if t.A == 0:
                        t.M = 0  # freeze (or mark as inert)
                        t.payload = ""  # compiler drops it
    # Compute effective top-tier envelope
    topA = max(i.A for i in instructions)
    P_res = 0
    M_res = 1
    for i in instructions:
        if i.A == topA:
            P_res |= i.P
            M_res &= i.M
```


Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487

```
# Emit only surviving payloads; attach resolved envelope
effective = [i for i in instructions if i.payload]
return effective, (topA, P_res, M_res)
```

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5476487